THE ALU REPORT

→The Arithmetic Logic Unit (ALU) serves as the computational core of the RISC201 processor, positioned exclusively within the Execute (EX) stage of both the 4-stage and 6-stage pipeline architectures. It operates as a purely combinational, stateless hardware block. Driven by control signals generated during the decode stage, the ALU consumes two 32-bit operands from the register file and resolves arithmetic, logical, memory-address calculation, and control-flow branch conditions within a single clock cycle, outputting a 32-bit result alongside derived operational flags (Zero, Negative, Carry, Overflow).

→To maximize pipeline throughput and simulate modern silicon performance, this ALU bypasses standard baseline logic in favor of highly optimized hardware algorithms.

→The internal datapath includes:

- **Kogge-Stone Parallel Prefix Adder: Handles all addition, subtraction, and address calculation using a parallel tree structure to reduce carry-propagation latency to $O(\log n)$.**
- **Radix-4 Modified Booth's Multiplier: Processes multiplier bits in overlapping triplets, halving the required cycles for signed multiplication.**
- **Non-Restoring Divider: Computes exact integer quotients and modulo remainders deterministically, eliminating the redundant clock cycles associated with standard restoring division.**
- **Logarithmic Barrel Shifter & Logic Arrays: Uses layered 2-to-1 multiplexers for constant-time bitwise shifts and fully parallel boolean arrays for logical operations.**

→The entire architecture is implemented at the structural gate level, strictly prohibiting the use of high-level C++ arithmetic operators (such as +, -, *, /). Foundational 1-bit boolean primitives—NOT, AND, OR, and XOR—are defined using explicit truth-table logic. These primary gates are cascaded across 32-bit wide bus structures (simulated via standard bitsets) to build complex physical paths. By calculating outputs entirely through the propagation of simulated electrical signals across these gate arrays, the software model accurately reflects real-world hardware behavior, combinational delays, and silicon-level binary arithmetic.

KOGGE-STONE PARALLEL PREFIX ADDER :

⇒Architecture of the 32-bit Kogge-Stone Adder→

The Kogge-Stone Adder (KSA) is a parallel prefix form of the carry-lookahead adder. It is widely considered the fastest adder design in high-performance VLSI architecture because it strictly bounds logic fan-out to 2 and computes the final carries in exactly $\log_2(N)$ stages. For a 32-bit architecture, this guarantees a deterministic logic depth of exactly 5 stages, allowing 32-bit addition to execute entirely within a single clock cycle.

The hardware operation is divided into three parallelized phases: Pre-processing, the Prefix Network, and Post-processing.

1. Pre-Processing Phase (Bitwise Generate and Propagate)

In the first phase, the adder simultaneously evaluates every bit pair (A_i, B_i) of the 32-bit operands to determine if that specific column will generate a carry-out or propagate a carry-in.

- **Generate (G_i): A carry is generated if both input bits are 1.**
  - $G_i = A_i \cdot B_i$
- **Propagate (P_i): A carry is propagated across the column if exactly one of the input bits is 1.**
  - $P_i = A_i \oplus B_i$
- **Because this step does not depend on any previous carries, all 32 Generate and Propagate signals are computed concurrently in $O(1)$ time.**

2. The Parallel Prefix Network (The Carry Tree)

This is the core routing matrix of the Kogge-Stone algorithm. It takes the initial G_i and P_i signals and combines them in a parallel binary tree to compute the actual carry signal for every bit position simultaneously.

The network operates in discrete stages. At each stage s, the logic at bit position i looks backward at a distance of 2^{s-1} to combine its current state with the state of the previous block.

The logic gates at each node evaluate the following prefix boolean equations:

- **Group Generate:**
  - $G_{i:j} = G_{i:k} + (P_{i:k} \cdot G_{k-1:j})$
- **Group Propagate:**
  - $P_{i:j} = P_{i:k} \cdot P_{k-1:j}$

For a 32-bit width, the hardware requires exactly $\log_2(32) = 5$ routing stages:

- **Stage 1: Each bit i combines with bit $i - 1$.**
- **Stage 2: Each bit i combines with bit $i - 2$.**
- **Stage 3: Each bit i combines with bit $i - 4$.**
- **Stage 4: Each bit i combines with bit $i - 8$.**
- **Stage 5: Each bit i combines with bit $i - 16$.**

By the end of Stage 5, the Group Generate signal at every index i effectively contains the final, fully-resolved carry-in signal (C_i) for that bit, factoring in the global C_{in} from the control unit. Because the wire distance doubles at each stage while the fan-out remains strictly at 2, the signal propagates at maximum electrical efficiency.

3. Post-Processing Phase (Sum and Flag Generation)
With all 32 carry bits (C_i) calculated simultaneously by the prefix network, the final addition relies on a single layer of XOR gates.

- **Final Sum (S_i): The sum bit is the XOR of the initial propagate signal and the carry-in from the previous bit.**
  - $S_i = P_i \oplus C_{i-1}$
- **Hardware Flags: The architecture directly taps the final nodes of the prefix network to output pipeline flags for the Execution (EX) stage:**
  - **Carry Out (C_{out}): Directly wired to C_{31}.**
  - **Overflow (V): Computed by XORing the final two carries: $V = C_{31} \oplus C_{30}$.**

```cpp
// ==========================================
// 4. KOGGE-STONE PARALLEL PREFIX ADDER
// ==========================================
bitset<32> ALU::ks_ADD(const bitset<32>& a, const bitset<32>& b, bool cin, bool& cout, bool& overflow) const {
    bool p[6][32];
    bool g[6][32];

    for (int i = 0; i < 32; ++i) {
        p[0][i] = gate_XOR(a[i], b[i]);
        g[0][i] = gate_AND(a[i], b[i]);
    }

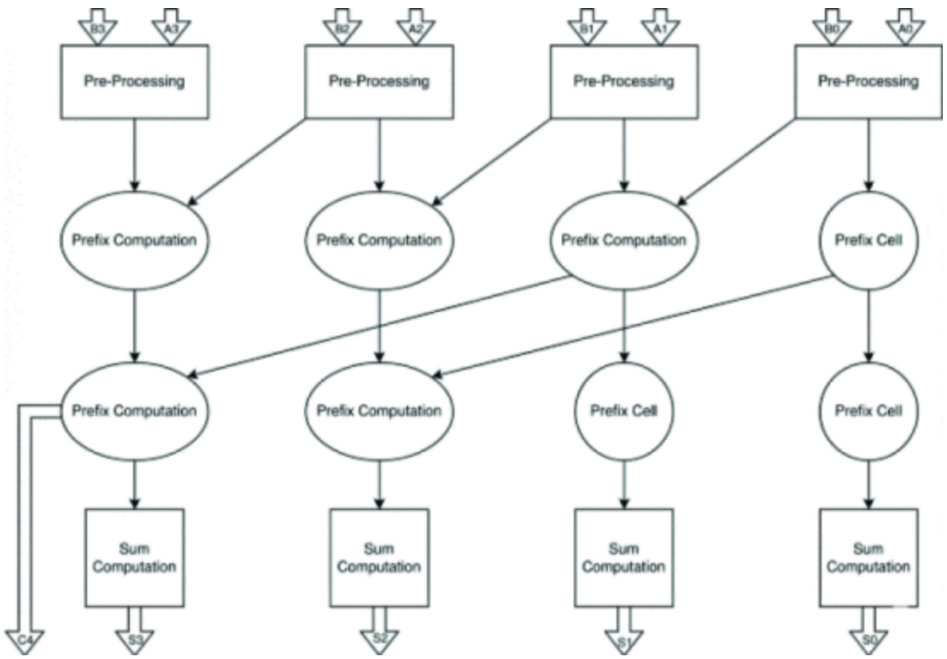
    for (int stage = 1; stage <= 5; ++stage) {
        int dist = 1 << (stage - 1);
        for (int i = 0; i < 32; ++i) {
            if (i >= dist) {
                g[stage][i] = gate_OR(g[stage-1][i], gate_AND(p[stage-1][i], g[stage-1][i - dist]));
                p[stage][i] = gate_AND(p[stage-1][i], p[stage-1][i - dist]);
            } else {
                g[stage][i] = g[stage-1][i];
                p[stage][i] = p[stage-1][i];
            }
        }
    }

    bitset<32> sum;
    bool c[32];
    for (int i = 0; i < 32; ++i) {
        bool prev_c = (i == 0) ? cin : c[i-1];
        c[i] = gate_OR(g[5][i], gate_AND(p[5][i], cin));
        sum[i] = gate_XOR(p[0][i], prev_c);
    }

    cout = c[31];
    overflow = gate_XOR(c[31], c[30]);
    return sum;
}

bitset<32> ALU::ks_SUB(const bitset<32>& a, const bitset<32>& b, bool& cout, bool& overflow) const {
    return ks_ADD(a, logic_NOT(b), true, cout, overflow);
}

bitset<32> ALU::twos_complement(const bitset<32>& a) const {
    bool dummy_cout, dummy_ovf;
    return ks_SUB(bitset<32>(0), a, dummy_cout, dummy_ovf);
}
```



NON-RESTORING DIVISION :

The Mechanics of Non-Restoring Division⇒

In a traditional "Restoring" divider, if a subtraction results in a negative partial remainder, the hardware must waste time "restoring" the previous value by adding the divisor back before moving to the next bit.

The Non-Restoring algorithm completely eliminates this wasted step. If the partial remainder becomes negative, the algorithm simply leaves it negative and sets the quotient bit to 0. To mathematically compensate for this "over-subtraction," the hardware simply adds the divisor in the very next cycle instead of subtracting it.

By alternating between subtraction and addition based on the sign bit, the algorithm executes in a strictly deterministic number of cycles (exactly N stages for an N-bit architecture), making it perfect for unrolling into a single-cycle combinational circuit.

The Algorithm Steps (Hardware Level)

Let Q be the Dividend, M be the Divisor, and A be the Accumulator (initialized to 0). N is the number of bits.

For each of the N stages, the hardware performs the following:

1. **Check Sign: Read the Most Significant Bit (MSB) of A *before* doing anything.**
2. **Shift: Shift the concatenated register pair (A, Q) left by 1 bit.**
3. **Add/Subtract:**
   - **If the original sign of A was 0 (Positive), compute $A = A - M$.**
   - **If the original sign of A was 1 (Negative), compute $A = A + M$.**
4. **Update Quotient: Set the Least Significant Bit (LSB) of Q to the inverted sign of the *new* A. ($Q_0 = \sim A_{MSB}$).**

Final Correction: After N stages, if A is negative, add M to A one last time to get the correct positive remainder.

Solved Example: 11 ÷ 3

To keep the trace manageable for documentation, we will scale the 32-bit architecture down to a 4-bit width ($N = 4$).

- **Dividend (Q): 1011 (11 in decimal)**
- **Divisor (M): 0011 (3 in decimal)**
- **Two's Complement of Divisor ($-M$): 1101**

Initial State: A = 0000, Q = 1011, M = 0011

Stage	Operation	Accumulator (A)	Quotient (Q)	Hardware Logic Explanation
Initial	Load registers	0000	1011	A is initialized to 0. Q holds dividend.
Stage 1	1. Shift Left (A, Q) 2. Sub M ($A = A + \sim M$) 3. Set $Q_0 = \sim A_{MSB}$	0001 1110 1110	011_ 011_ 0110	Original A was Positive (MSB=0). 0001 + 1101 = 1110. New A is Negative (MSB=1), so $Q_0 = 0$.
Stage 2	1. Shift Left (A, Q) 2. Add M ($A = A + M$) 3. Set $Q_0 = \sim A_{MSB}$	1100 1111 1111	110_ 110_ 1100	Original A was Negative (MSB=1). 1100 + 0011 = 1111. New A is Negative (MSB=1), so $Q_0 = 0$.

Stage 3	**1. Shift Left** (A, Q) **2. Add** M **(**$A = A + M$**)** **3. Set** $Q_0 =\sim A_{MSB}$	11z1 0010 0010	100_ 100_ 1001	**Original** A **was Negative (MSB=1).** 1111 + 0011 = 0010 **(carry ignored).** **New** A **is Positive (MSB=0), so** $Q_0 = 1$.
Stage 4	**1. Shift Left** (A, Q) **2. Sub** M **(**$A = A +\sim M$ **)** **3. Set** $Q_0 =\sim A_{MSB}$	0101 0010 0010	001_ 001_ 0011	**Original** A **was Positive (MSB=0).** 0101 + 1101 = 0010 **(carry ignored).** **New** A **is Positive (MSB=0), so** $Q_0 = 1$.
Final	**Check Correction**	0010	0011	A **is positive (MSB=0). No correction needed.**

Final Results Verified:

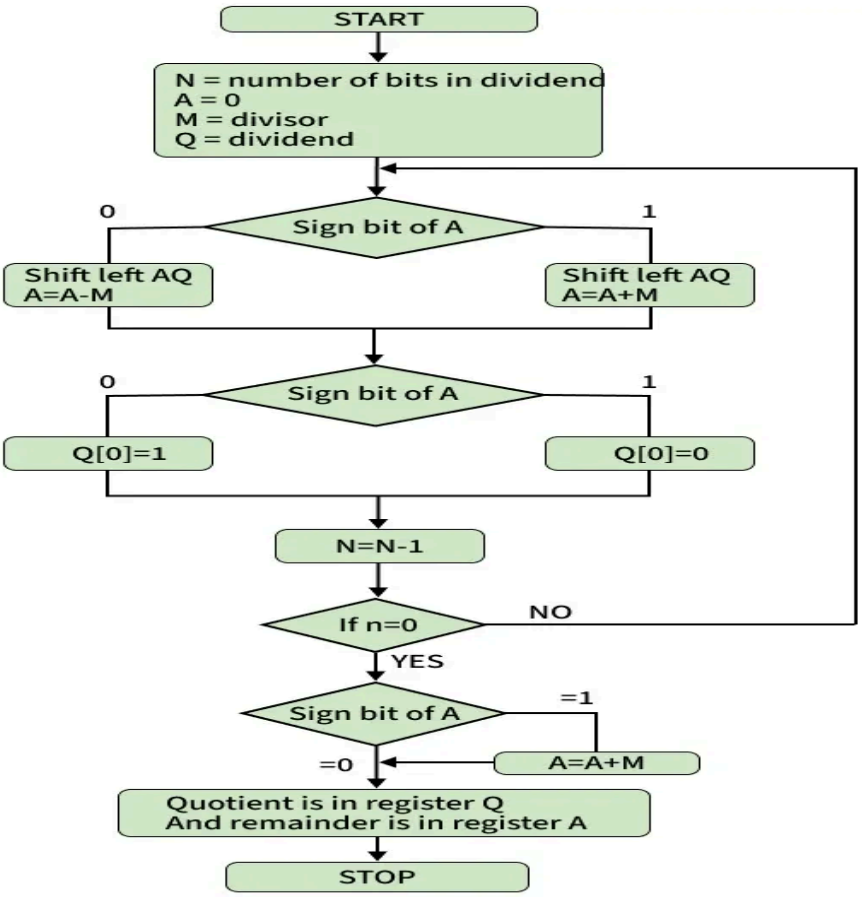
- **Quotient (**Q**): 0011 → 3**
- **Remainder (**A**): 0010 → 2**
- **Math Check:** $11 = (3 \times 3) + 2$**. The hardware logic perfectly resolves the division in exactly** N **passes.**

Handling Signed Two's Complement Division (The Wrapper Approach)

Building a divider that natively processes two's complement negative numbers at every stage (such as SRT division) requires complex Lookup Tables (LUTs) and redundant signed-digit conversion. This drastically increases silicon area and gate delay, threatening the RISC201's single-cycle execution constraint.

To maintain speed and hardware efficiency, the ALU implements an Absolute Value Wrapper around an optimized unsigned Non-Restoring core. This approach maximizes hardware reusability by routing data through the existing Kogge-Stone subtractor rather than building dedicated signed-division logic.

- **Pre-Processing (Absolute Value): The hardware inspects the Most Significant Bit (bit 31) of both the dividend and divisor. If an operand is negative (MSB = 1), it is routed through the Kogge-Stone subtractor to compute its positive two's complement absolute value.**
- **Core Execution: The 32-stage Non-Restoring Divider processes these purely unsigned values deterministically, without requiring dynamic sign-extension logic or risking asymmetric edge-case faults.**
- **Post-Processing (Sign Restoration): The ALU applies standard mathematical formatting to the output.**
    - **For Division (`DIV`): The hardware XORs the original sign bits of the operands. If the signs differed, the resulting quotient is routed back through the Kogge-Stone subtractor to restore its two's complement negative form.**
    - **For Modulo (`MOD`): By architectural convention, the remainder must match the sign of the original dividend. If the original dividend was negative, the remainder is routed through the subtractor to become negative.**




```cpp
pair<bitset<32>, bitset<32>> ALU::algo_NonRestoringDiv(const bitset<32>& dividend, const bitset<32>& divisor) const {
    bitset<32> A = 0, Q = dividend, M = divisor;
    bool dummy_cout, dummy_ovf;

    for (int i = 0; i < 32; ++i) {
        bool a_sign = A[31];

        A = shift_LSL(A, 1);
        A[0] = Q[31];
        Q = shift_LSL(Q, 1);

        if (a_sign == false) A = ks_SUB(A, M, dummy_cout, dummy_ovf);
        else                 A = ks_ADD(A, M, false, dummy_cout, dummy_ovf);

        Q[0] = gate_NOT(A[31]);
    }

    if (A[31] == true) A = ks_ADD(A, M, false, dummy_cout, dummy_ovf);
    return {Q, A};
}
```

RADIX–4 BOOTH'S MULTIPLIER

Architecture of the Radix-4 Modified Booth Multiplier⇒

While basic shift-and-add multiplication is structurally simple, it generates N partial products for an N-bit multiplier. For a 32-bit architecture, adding 32 partial products creates a massive adder tree that severely limits maximum clock frequency. To meet the strict single-cycle combinational constraints of the RISC201 pipeline, our ALU implements a Radix-4 Modified Booth's Algorithm.

1. The Radix-4 Optimization (Halving the Partial Products)

The core advantage of Radix-4 Booth encoding is that it inspects the multiplier in overlapping 3-bit windows rather than processing it one bit at a time. By shifting the window by 2 bits per stage (instead of 1), the algorithm processes two bits of the multiplier simultaneously.

This mathematical optimization strictly halves the number of partial products required. For our 32-bit ALU, it reduces the addition tree from 32 layers down to just 16. This *50%* reduction in logic depth drastically decreases gate delay and silicon area.

2. Native Two's Complement Execution

Unlike our division algorithm (which requires an absolute-value wrapper to handle negative numbers), Booth's Algorithm is mathematically designed to operate natively on two's complement numbers. The algorithm inherently recognizes strings of 1s as negative weights, allowing it to correctly multiply any combination of positive and negative operands without requiring pre- or post-processing stages.

3. The 3-Bit Overlapping Window

Let the Multiplicand be A and the Multiplier be M.

To begin, the hardware appends a virtual zero to the right of the Multiplier's Least Significant Bit (LSB) to serve as the initial reference point ($M_{-1} = 0$).

For each of the 16 stages, the encoding logic looks at a 3-bit window consisting of M_{i+1}, M_i, and M_{i-1}. Based on this pattern, a specific multiple of the Multiplicand (A) is selected and added to the accumulating product.

4. Hardware Recoding Logic (The Truth Table)

Depending on the 3-bit window, the hardware generates one of five possible partial products: $0, +A, -A, +2A,$ **or** $-2A$**.**

Mi+1 (Left)	Mi (Middle)	Mi−1 (Right)	Operation	Hardware Logic Execution
0	**0**	**0**	0	**Output all zeros.**
0	**0**	**1**	$+A$	**Pass A straight through.**
0	**1**	**0**	$+A$	**Pass A straight through.**
0	**1**	**1**	$+2A$	**Left-shift A by 1 bit (Hardwired).**
1	**0**	**0**	$-2A$	**Left-shift A by 1 bit, then invert and add 1.**
1	**0**	**1**	$-A$	**Invert A and add 1 (Two's Complement).**
1	**1**	**0**	$-A$	**Invert A and add 1 (Two's Complement).**
1	**1**	**1**	0	**Output all zeros.**

5. Why this is highly efficient in Silicon

At first glance, generating $\pm 2A$ or $-A$ seems like it requires additional arithmetic hardware, but at the physical silicon level, it requires zero active addition or multiplication:

- **Generating $+2A$: Multiplying by 2 in binary is just a left shift by 1. The hardware does not compute this; it simply routes the wires of A one position to the left as they enter the multiplexer.**
- **Generating $-A$: To generate a negative version of A, the hardware simply routes A through NOT gates (XORing with 1) and sets the carry-in bit (C_{in}) of the subsequent adder to 1.**
- **Generating $-2A$: This is just a combination of the two physical wire tricks above (shifted left by 1 wire, inverted, with $C_{in} = 1$).**

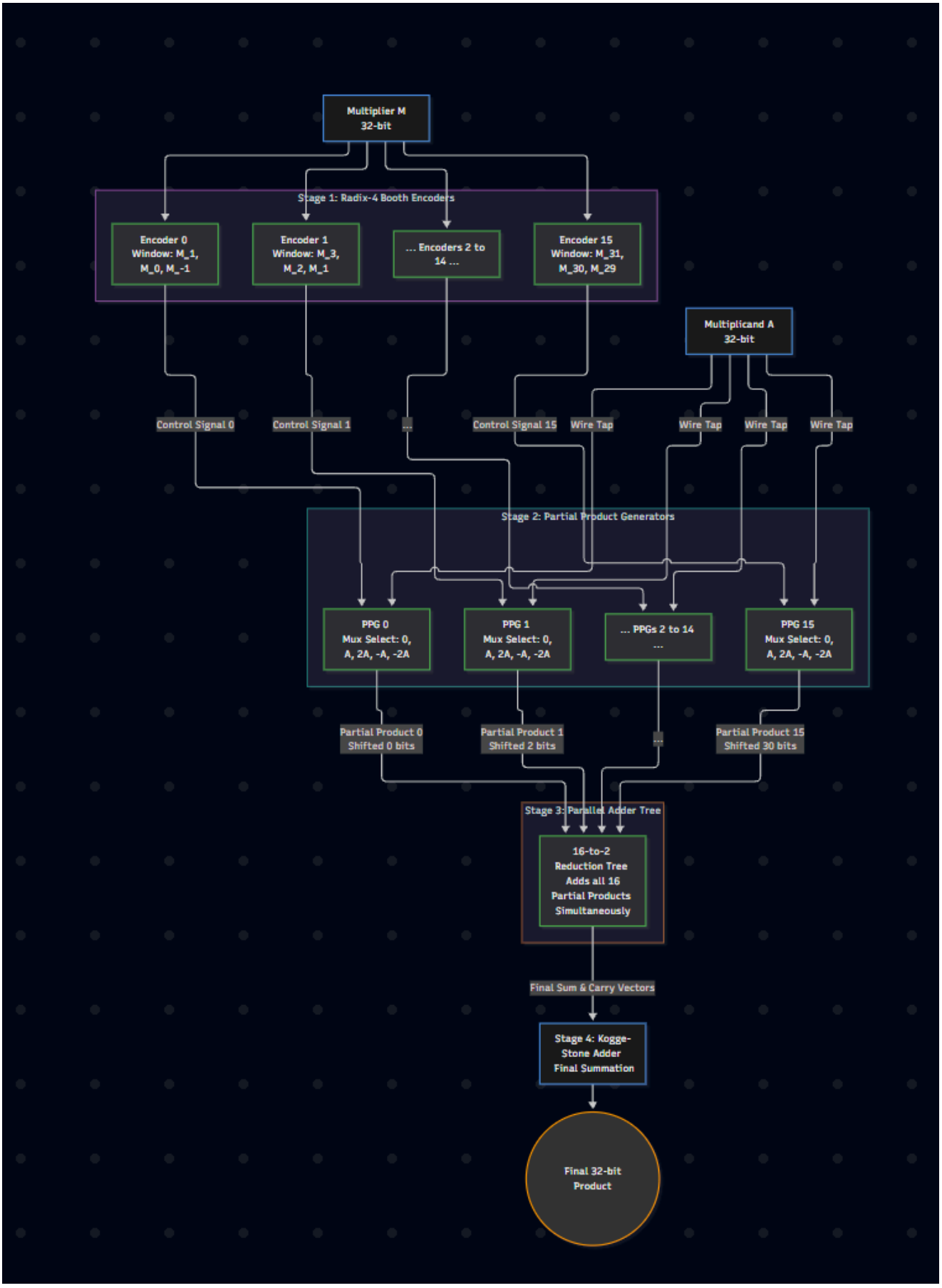
By using physical routing and simple inversion gates to generate all partial products, the hardware instantly sets up the 16 layers, leaving only the final summation to the parallel adder tree.

```
bitset<32> ALU::algo_Radix4BoothMul(const bitset<32>& a, const bitset<32>& b) const {
    bitset<32> A = 0;
    bitset<32> Q = b;
    bitset<32> M = a;
    bool q_minus_1 = false, dummy_cout, dummy_ovf;

    for (int i = 0; i < 16; ++i) {
        uint8_t window = (Q[1] << 2) | (Q[0] << 1) | q_minus_1;
        bitset<32> addend = 0;

        switch (window) {
            case 1: case 2: addend = M; A = ks_ADD(A, addend, false, dummy_cout, dummy_ovf); break;
            case 3:         addend = shift_LSL(M, 1); A = ks_ADD(A, addend, false, dummy_cout, dummy_ovf); break;
            case 4:         addend = shift_LSL(M, 1); A = ks_SUB(A, addend, dummy_cout, dummy_ovf); break;
            case 5: case 6: addend = M; A = ks_SUB(A, addend, dummy_cout, dummy_ovf); break;
            case 0: case 7: break;
        }

        q_minus_1 = Q[1];
        bool a_lsb_0 = A[0], a_lsb_1 = A[1];
        Q = shift_LSR(Q, 2);
        Q[31] = a_lsb_1; Q[30] = a_lsb_0;
        A = shift_ASR(A, 2);
    }
    return Q;
}
```


Multiplier M
32-bit
Stage 1: Radix-4 Booth Encoders
Encoder 0
Window: M_1,
M_0, M_-1
Encoder 1
Window: M_3,
M_2, M_1
... Encoders 2 to
14 ...
Encoder 15
Window: M_31,
M_30, M_29
Multiplicand A
32-bit
Control Signal 0
Control Signal 1
...
Control Signal 15
Wire Tap
Wire Tap
Wire Tap
Wire Tap
Stage 2: Partial Product Generators
PPG 0
Mux Select: 0,
A, 2A, -A, -2A
PPG 1
Mux Select: 0,
A, 2A, -A, -2A
... PPGs 2 to 14
...
PPG 15
Mux Select: 0,
A, 2A, -A, -2A
Partial Product 0
Shifted 0 bits
Partial Product 1
Shifted 2 bits
...
Partial Product 15
Shifted 30 bits
Stage 3: Parallel Adder Tree
16-to-2
Reduction Tree
Adds all 16
Partial Products
Simultaneously
Final Sum & Carry Vectors
Stage 4: Kogge-
Stone Adder
Final Summation
Final 32-bit
Product

Native Signed Execution via Radix-4 Booth's Multiplier

Unlike division, the multiplier requires absolutely no pre-processing or post-processing wrappers to handle negative numbers. The Radix-4 Modified Booth's Algorithm is mathematically designed to operate natively on two's complement signed integers.

Standard shift-and-add multipliers fail on negative numbers because they cannot inherently distinguish between a high-magnitude unsigned integer and a negative two's complement integer. Radix-4 Booth encoding solves this entirely within its core logic:

- **Sign-Aware Encoding: By evaluating overlapping 3-bit windows, the Booth encoders naturally recognize continuous strings of 1s. In two's complement arithmetic, a string of leading 1s represents a negative weight.**
- **Inherent Subtraction: When the encoders detect these negative-weight patterns, they automatically recode the operation from an addition into a subtraction (selecting $-A$ or $-2A$ for the partial product).**
- **Seamless MSB Processing: The final encoder window at the top of the 32-bit word (M_{31}, M_{30}, M_{29}) inherently factors the multiplier's sign bit (M_{31}) into the final partial product.**

By pushing the sign logic directly into the 3-bit recoding truth table, the hardware natively outputs a perfect 32-bit two's complement product without requiring a single extra clock cycle or corrective hardware routing.

LOGARITHMIC BARREL SHIFT

Architecture of the 32-bit Logarithmic Barrel Shifter

In a standard microprocessor, shifting data using linear shift registers requires a clock cycle for every bit shifted (up to 31 cycles for a 32-bit word). This sequentially blocks the execution stage. To meet the strict single-cycle constraint of the RISC201 architecture, the ALU implements a Logarithmic Barrel Shifter.

This design achieves an $O(1)$ combinational delay by physically cascading multiplexers, completing any shift from 0 to 31 positions instantly via static wire routing.

1. The Logarithmic Multiplexer Network

The shifter does not compute the shift; it physically routes signals through a grid of $2{:}\,1$ multiplexers. For a 32-bit architecture, the hardware requires exactly $\log_2(32) = 5$ sequential layers of multiplexers.

The 5-bit shift amount (shift_amt[4:0]) acts as the control bus. Each individual bit of this bus is hardwired to the "select" pins of one entire multiplexer layer:

- **Stage 0 (Shift by 1): Controlled by shift_amt[0]. Routes data left/right by 1 wire, or passes it straight through.**
- **Stage 1 (Shift by 2): Controlled by shift_amt[1]. Routes data by 2 wires, or passes it straight through.**
- **Stage 2 (Shift by 4): Controlled by shift_amt[2]. Routes data by 4 wires, or passes it straight through.**
- **Stage 3 (Shift by 8): Controlled by shift_amt[3]. Routes data by 8 wires, or passes it straight through.**
- **Stage 4 (Shift by 16): Controlled by shift_amt[4]. Routes data by 16 wires, or passes it straight through.**

Because the shift distances are powers of two, any shift amount from 0 to 31 can be constructed by combining these discrete stages.

2. The Physical Routing (Zero Active Computation)

At the silicon level, shifting data by N positions requires no active logic gates (like AND/OR/XOR). It is achieved purely by skewed wiring.

When a multiplexer in Stage 2 (Shift by 4) is set to 1, its input pin is physically soldered to the wire coming from position $i \pm 4$ of the previous stage. The

multiplexers merely act as switches, instantly opening a physical electrical path from the source bit to its final destination bit across the 5 layers.

3. Operation Support: LSL, LSR, and ASR

The RISC201 ISA demands three distinct shift operations (LSL, LSR, ASR). The barrel shifter handles these entirely through the routing of the "padding" wires that fill the empty spaces left behind by a shift:

- Logical Shift Left (LSL): The hardware shifts the data matrix left. Any multiplexer input that reads past the right-most bit (index < 0) is physically grounded to 0.
- Logical Shift Right (LSR): The hardware shifts the data matrix right. Any multiplexer input that reads past the left-most bit (index > 31) is physically grounded to 0.
- Arithmetic Shift Right (ASR): Used for signed two's complement numbers. Instead of grounding the out-of-bounds wires to 0, the hardware physical fan-outs (splits) the Most Significant Bit (MSB, the sign bit) of the original operand. Any multiplexer reading past index 31 reads directly from this MSB wire, perfectly preserving the sign of negative numbers.

4. Execution Trace: Shifting by 13

To demonstrate the logarithmic cascading, consider an instruction shifting a word by 13 positions.

The binary representation of 13 is 01101. The hardware resolves this instantly across the 5 stages:

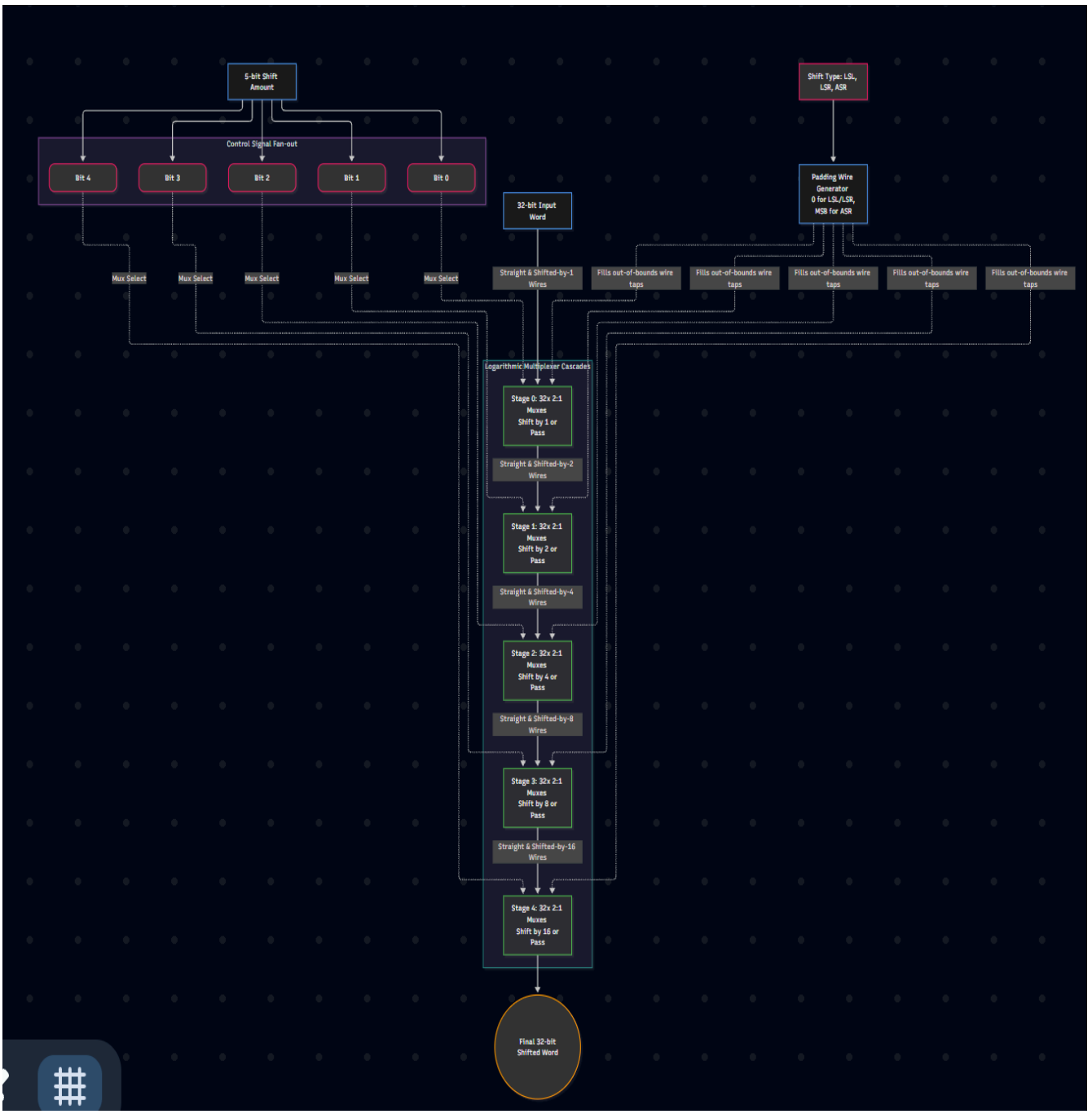
1. Stage 0 (bit 0 = 1): Multiplexers select the shifted wire. (Total shift: 1)
2. Stage 1 (bit 1 = 0): Multiplexers select the straight wire. (Total shift: 1 + 0 = 1)
3. Stage 2 (bit 2 = 1): Multiplexers select the shifted wire. (Total shift: 1 + 4 = 5)
4. Stage 3 (bit 3 = 1): Multiplexers select the shifted wire. (Total shift: 5 + 8 = 13)
5. Stage 4 (bit 4 = 0): Multiplexers select the straight wire. (Total shift: 13 + 0 = 13)

By the time the electrical signal exits Stage 4, it has been physically displaced by exactly 13 wires in a single combinational pass, ready to be latched by the bus.

```cpp
// ========================================
// 3. LOGARITHMIC BARREL SHIFTERS
// ========================================
bitset<32> ALU::shift_LSL(const bitset<32>& a, uint32_t shift_amt) const {
    bitset<32> s[6];
    s[0] = a;
    for (int stage = 0; stage < 5; ++stage) {
        int offset = 1 << stage;
        bool do_shift = (shift_amt >> stage) & 1;
        for (int i = 0; i < 32; ++i) {
            s[stage+1][i] = do_shift ? (i >= offset ? s[stage][i - offset] : false) : s[stage][i];
        }
    }
    return s[5];
}

bitset<32> ALU::shift_LSR(const bitset<32>& a, uint32_t shift_amt) const {
    bitset<32> s[6];
    s[0] = a;
    for (int stage = 0; stage < 5; ++stage) {
        int offset = 1 << stage;
        bool do_shift = (shift_amt >> stage) & 1;
        for (int i = 0; i < 32; ++i) {
            s[stage+1][i] = do_shift ? (i + offset < 32 ? s[stage][i + offset] : false) : s[stage][i];
        }
    }
    return s[5];
}

bitset<32> ALU::shift_ASR(const bitset<32>& a, uint32_t shift_amt) const {
    bitset<32> s[6];
    s[0] = a;
    bool sign = a[31];
    for (int stage = 0; stage < 5; ++stage) {
        int offset = 1 << stage;
        bool do_shift = (shift_amt >> stage) & 1;
        for (int i = 0; i < 32; ++i) {
            s[stage+1][i] = do_shift ? (i + offset < 32 ? s[stage][i + offset] : sign) : s[stage][i];
        }
    }
    return s[5];
}
```


5-bit Shift Amount
Shift Type: LSL, LSR, ASR
Control Signal Fan-out
Bit 4
Bit 3
Bit 2
Bit 1
Bit 0
Padding Wire Generator 0 for LSL/LSR, MSB for ASR
32-bit Input Word
Mux Select
Mux Select
Mux Select
Mux Select
Mux Select
Straight & Shifted-by-1 Wires
Fills out-of-bounds wire taps
Fills out-of-bounds wire taps
Fills out-of-bounds wire taps
Fills out-of-bounds wire taps
Fills out-of-bounds wire taps
Logarithmic Multiplexer Cascades
Stage 0: 32x 2:1 Muxes Shift by 1 or Pass
Straight & Shifted-by-2 Wires
Stage 1: 32x 2:1 Muxes Shift by 2 or Pass
Straight & Shifted-by-4 Wires
Stage 2: 32x 2:1 Muxes Shift by 4 or Pass
Straight & Shifted-by-8 Wires
Stage 3: 32x 2:1 Muxes Shift by 8 or Pass
Straight & Shifted-by-16 Wires
Stage 4: 32x 2:1 Muxes Shift by 16 or Pass
Final 32-bit Shifted Word

CHOICE OF ALGORITHMS :

Q. WHY KOGGE-STONE OVER RCA, CLA, OR ANY OTHER ADDERS?

-> Ripple-Carry Adder (RCA): The RCA has an $O(N)$ time complexity because the carry bit must sequentially ripple through every single full-adder block. For a 32-bit architecture, this creates a 32-gate propagation delay, which is far too slow to execute within the strict single-cycle Execute (EX) stage constraint of the RISC201 pipeline.

-> Carry Lookahead Adder (CLA): While a standard CLA computes carries faster than an RCA, its textbook architecture scales poorly to 32 bits. As the bit width increases, the logic gates calculating the highest-order carries must take inputs from every single preceding bit. This creates massive electrical fan-out and requires logic gates with up to 32 inputs. In physical silicon, this extreme fan-out severely degrades signal propagation speed.

-> Kogge-Stone Parallel Prefix Adder: The Kogge-Stone algorithm solves the scaling bottlenecks of both the RCA and CLA. It computes generate and propagate signals in a parallel binary tree structure that strictly limits logic fan-out to exactly 2 at any given node. It mathematically guarantees that all 32 final carries are resolved in exactly $\log_2(32) = 5$ uniform logic stages. This provides the absolute lowest theoretical latency possible for 32-bit addition, ensuring the processor's clock cycle is never stalled by arithmetic operations.

Q. WHY RADIX 4 BOOTH'S ALGORITHM OVER RADIX 2 OR ANY OTHER MULTIPLIER?

-> Standard Shift-and-Add (Array) Multipliers: Basic multipliers cannot natively handle signed two's complement numbers without requiring extra absolute-value hardware wrappers. Furthermore, multiplying two 32-bit numbers generates 32 individual partial products. Summing a 32-layer deep addition tree in a single clock cycle creates a massive combinational delay that would severely bottleneck the processor's maximum clock speed.

-> Radix-2 Booth's Algorithm (Textbook): While standard Radix-2 natively handles signed numbers by looking at 2-bit windows, it does not guarantee a reduction in partial products. In the worst-case hardware scenario (an alternating bit pattern like `01010101`), Radix-2 still generates a full 32 partial products. Because hardware must be built to accommodate the worst-case scenario, the silicon reduction tree must still be physically built 32 layers deep, offering no actual speed or area improvements over a standard array multiplier.

-> Radix-4 Modified Booth's Algorithm: By inspecting overlapping 3-bit windows and shifting by 2 bits at a time, Radix-4 mathematically guarantees that the number of partial products is strictly halved (from 32 down to 16). This 50% reduction cuts the depth of the parallel adder tree in half. It drastically decreases gate delay and saves massive amounts of silicon area, allowing the multiplier to comfortably execute within the RISC201's single-cycle Execute (EX) limit while natively processing signed integers.

Q. WHY NON-RESTORING DIVISION OVER RESTORING, GOLDSCHMIDT, OR SRT DIVISION?

-> Restoring Division: In a Restoring divider, if a subtraction results in a negative partial remainder, the hardware must execute a "restore" step by adding the divisor back to fix it. This creates a data-dependent bottleneck. In a purely combinational circuit, building hardware to conditionally undo its own math requires extra multiplexers and wastes severe gate delay, making it too slow and inefficient for a modern single-cycle ALU.

-> Goldschmidt Division (Iterative Approximation): Goldschmidt (along with Newton-Raphson) uses a series of iterative multiplications to rapidly converge on an approximate quotient. While heavily used in pipelined Floating-Point Units (FPUs), it is completely unviable for this ALU. It requires routing data through a massive hardware multiplier multiple times over several clock cycles, and it requires highly complex rounding logic to guarantee the exact integer modulo remainders required by the RISC201 ISA.

-> SRT Division (Sweeney, Robertson, and Tocher): SRT is the industry standard for commercial high-speed CPUs (like Intel and AMD). It looks at multiple bits at once and skips over strings of zeros. However, SRT requires physical hardware Lookup Tables (LUTs) to estimate quotient digits on the fly, and it outputs the result in a complex "Redundant Signed-Digit" format. Converting that bizarre format back into a standard 32-bit integer requires an entire secondary Kogge-Stone adder at the end of the pipeline. It is far too area-expensive and structurally complex for this specific processor scope.

-> Non-Restoring Division: This algorithm strikes the perfect balance for the RISC201 architecture. It mathematically eliminates the wasted "restore" step entirely by simply alternating between subtraction and addition on the next cycle. This guarantees a mathematically pure, strictly deterministic execution in exactly 32 uniform hardware stages. It allows the integer division to be unrolled cleanly into a single-cycle combinational circuit without the massive lookup-table overhead of SRT or the wasted gate delay of a Restoring divider.

Q. WHY LOGARITHMIC BARREL SHIFTER OVER LINEAR SHIFT REGISTERS OR CROSSBAR SHIFTERS?

-> Linear Shift Registers (Sequential): A standard linear shift register moves data by only one bit position per clock cycle. Executing a maximum 31-bit shift would require the CPU to stall for 31 sequential clock cycles. This completely breaks the RISC201 pipeline design, as the ALU is strictly required to execute all arithmetic and logical instructions within a single, combinational Execution (EX) stage.

-> Crossbar / Matrix Barrel Shifter: A traditional single-stage matrix shifter can perform any 32-bit shift in a single cycle by routing the signal through a massive 32×32 grid of transmission gates. However, this requires 1,024 individual hardware switches. This sprawling layout consumes an enormous amount of physical silicon area and introduces heavy capacitive loading on the wires, which degrades the electrical signal and slows down the maximum clock frequency.

-> Logarithmic Barrel Shifter: This architecture provides the optimal balance between execution speed and silicon area. By cascading standard 2-to-1 multiplexers in discrete stages based on powers of two (shifting by 1, 2, 4, 8, and 16), it achieves the exact same instant, single-cycle combinational delay as the massive crossbar matrix. However, because it relies on exactly $\log_2(32) = 5$ routing layers, it reduces the hardware footprint from 1,024 switches down to just 160 multiplexers. This massive area savings is achieved without sacrificing any pipeline timing performance.